

TECHNICAL ARCHITECTURE v4.0

QUMANITY

TECHNICAL ARCHITECTURE DOCUMENT

Complete Engineering Specification for the
Quantum-Informed Governance Protocol

Internal / Developer Reference | Companion: White Paper v4.0

SITA Foundation (Sekyor Intelligence Tantra Foundation)

July 2026 | Classification: Internal / Developer Reference

Web: <https://qumanity.in> | **GitHub:** github.com/sitaram-box/QUMANITY

Contact: founder@qumanity.in

Table of Contents

1. Document Purpose

2. System Overview

2.1 Technology Stack

2.2 Repository Structure

2.3 Configuration

3. Architecture Layers

3.1 Three-Layer Model

3.2 Request Flow

4. Identity System

4.1 Dual-Key Model

4.2 ID Format Specification

4.3 Recovery Mechanism

4.4 User Types and Permissions

5. Geographic Hierarchy

5.1 Hierarchy Levels

5.2 Collective Boards

6. Four Timelines Implementation

7. Governance Engine

7.1 Post Escalation

7.2 Zodiac Elections

7.3 Council Roles

7.4 Varna System

8. Qoin Economy

8.1 Denominations

8.2 Greedy Algorithm

8.3 Weekly Settlement

9. Database Schema

9.1 User Management

9.2 Content and Social

9.3 Economy

9.4 Governance

10. API Reference

11. Core Modules

12. PLNN Specification

13. Security Model

14. Deployment

15. Feature Implementation Status

16. Legal and Compliance

17. Budget Breakdown

18. Migration Paths

1. Document Purpose

This document contains the complete technical specifications for the Qumanity platform. It serves as the primary engineering companion to the Qumanity White Paper v4.0, providing the implementation detail necessary for developers, system architects, security auditors, and DevOps engineers to understand, build upon, and deploy the protocol.

This document includes specifications not present in the White Paper:

- Complete database schemas with column types, constraints, and indexes
- Full API endpoint catalogue with method signatures and response formats
- PLNN node architecture and sync protocol specifications
- Security implementation details, threat models, and mitigation strategies
- Code module descriptions and dependency graphs
- Deployment runbooks for development, staging, and production environments
- Legal analysis framework for compliance across jurisdictions
- Detailed budget breakdown for pilot and scale phases
- Team role specifications and skill requirements
- Migration paths from existing systems and between Qumanity versions

2. System Overview

2.1 Technology Stack

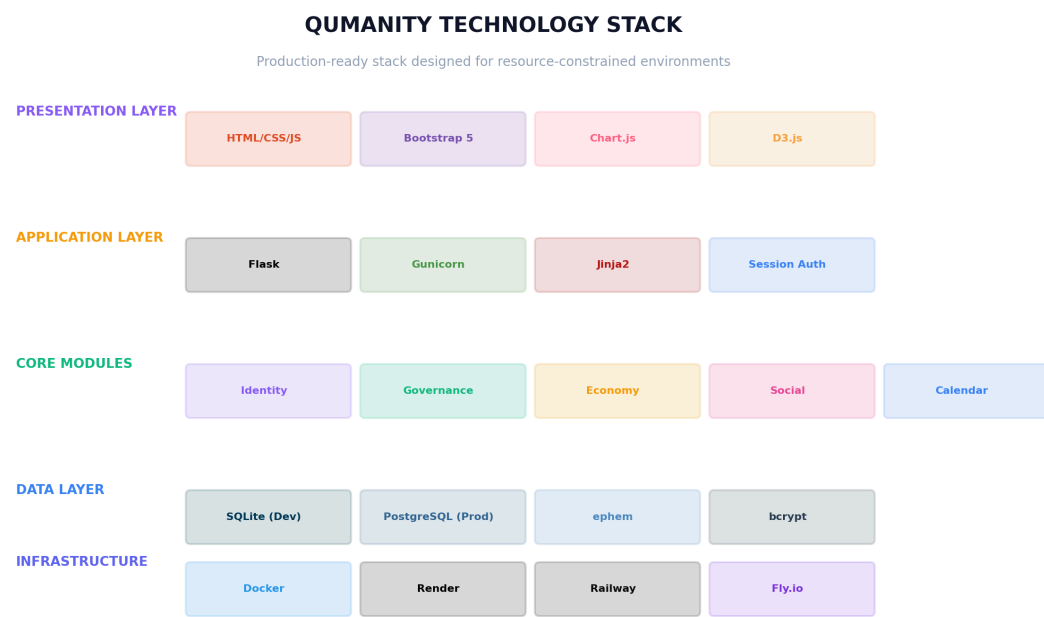


Figure 1: Qumanity Technology Stack — Full component diagram

Table 1 Technology Stack Specification

Layer	Technology	Version / Notes
Runtime	Python	3.12+ (Docker image: 3.14-slim)
Web Framework	Flask	Served by Gunicorn in production
Database (Dev)	SQLite	indiaq.db with WAL mode
Database (Prod)	PostgreSQL	Via DATABASE_URL connection string
Frontend	HTML, CSS, JavaScript	Bootstrap 5, Chart.js, D3.js
Configuration	config.py	Env-driven via python-dotenv
Container	Docker	Non-root quantumuser, HEALTHCHECK
Platforms	Render, Railway, Fly.io	See deployment section
i18n	translations.py	12 Indian languages supported
Astrology	ephem (optional)	Sidereal Lahiri; fallback approximation
Blockchain (Phase 3)	Polygon	Schema stubs in blockchain_core.py
Mobile (Phase 4)	Flutter / SwiftUI	iOS / macOS native application

2.2 Repository Structure

```

Qumanity/
|-- app.py                # Main Flask application (11,000+ lines)
|-- config.py             # Centralized configuration
|-- qoin_core.py          # Qoin economy, settlement engine
|-- identity_core.py      # Dual-key IDs, OTP recovery
|-- karma_core.py         # Karma <-> Varna integration
|-- planetary_core.py     # Birth chart, ephemeris (Akasha)
|-- birth_chart.py        # Astrological calculations
|-- varna_core.py         # Vidya/Raksha/Artha/Seva scoring
|-- leadership_core.py    # Council management
|-- blockchain_core.py    # Blockchain schema stubs
|-- language_core.py      # Multi-language support
|-- donation_core.py      # Donation flows
|-- social_core.py        # Connections, messaging
|-- element_core.py       # Five elements logic
|-- global_core.py        # Global account statistics
|-- deceased_core.py      # Deceased user handling
|-- referral_core.py      # Referral system
|-- sita_platform_core.py # SITA platform integration
|-- scheduler.py          # Weekly settlement cron
|-- init_db.py            # Database initialization
|-- templates/            # Jinja2 HTML templates
|-- static/               # CSS, JS, images, poems.json
|-- knowledge/            # Stage contracts & references
|-- docs/                 # White paper, architecture, context
|-- Dockerfile
|-- docker-compose.yml
|-- render.yaml
|-- railway.json
|-- requirements.txt

```

2.3 Configuration

All settings flow through `config.py`. Do not scatter `os.environ.get` calls across the codebase.

Table 2 Required Production Variables

Variable	Purpose
FLASK_ENV	Set to <code>production</code>
SECRET_KEY	Flask session encryption (48+ char random)
QOIN_WALLET_ENCRYPTION_KEY	Wallet data encryption (Fernet/AES)
DATABASE_URL	PostgreSQL connection string

Table 3 Optional Configuration Variables

Variable	Purpose
MAIL_SERVER , MAIL_PORT , MAIL_USE_TLS	SMTP configuration for OTP/recovery
MAIL_USERNAME , MAIL_PASSWORD	Email credentials
RAZORPAY_KEY_ID , RAZORPAY_KEY_SECRET	Payment gateway credentials
DONATION_UPI_VPA	UPI ID for donation processing
ENABLE_BLOCKCHAIN	Toggle blockchain integration
MASTER_KEY	Emergency admin access key

`config.validate()` and `config.log_warnings()` run at startup to ensure all required variables are present.

3. Architecture Layers

3.1 Three-Layer Model

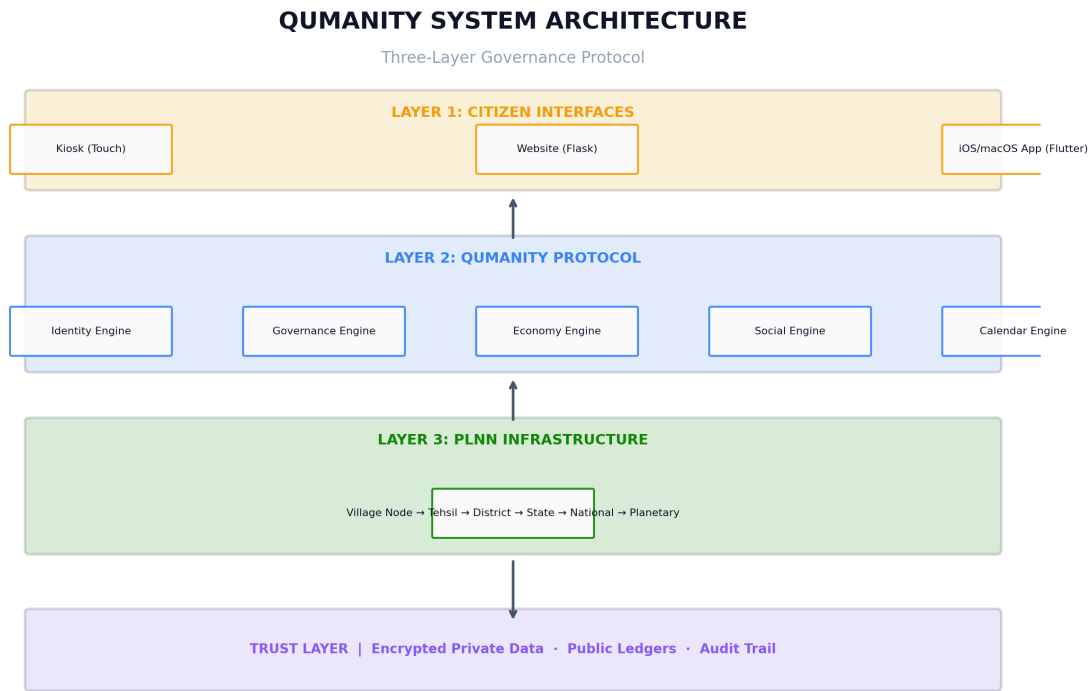


Figure 2: Three-Layer Architecture: Citizen Interfaces, Application Protocol, PLNN Infrastructure

Table 4 Architecture Layer Responsibilities

Layer	Components	Responsibilities
Layer 1: Citizen Interfaces	Kiosk (touch), Website (Flask), iOS/macOS App (Flutter)	Authentication, content display, vote capture, karma claim submission
Layer 2: Application	app.py + core modules	Identity, Governance, Economy, Social, Calendar, Admin
Layer 3: PLNN	Village → Tehsil → District → State → National → Planetary	Distributed data, offline sync, blockchain anchor
Data Layer	SQLite / PostgreSQL + Encrypted Wallets	Persistent storage, query optimization, backup

3. Geographic scope filtered by `current_location_id`
4. Business logic delegated to `*_core.py` modules
5. Database read/write via SQLite/PostgreSQL
6. JSON response (API) or Jinja2 render (pages)
7. Pending transactions queued for weekly settlement

4. Identity System

4.1 Dual-Key Model

[illegible]

Figure 3: Dual-Key Identity: Private ID (immutable) and Public ID (mutable on relocation)

Table 5 Identity Key Properties

Property	Private ID	Public ID
Field	private_id	public_id
Basis	Birth location + astrology	Present location
Mutability	Immutable (never changes)	Mutable (updates on relocation)
Scope	Lifelong identity, karma, wallet	Voting, council eligibility
Privacy	Encrypted, citizen-only	Visible on public boards

4.2 ID Format Specification

Format: [RandomPrefix]-[FirstInitial][LastInitial]-[GenderCode][AgeCode]-[ElementCode]
[SignCode]-[LocationPath]

Table 6 ID Component Specification

Component	Values	Example
RandomPrefix	3-4 alphanumeric characters	X7K
GenderCode	M, F, O (Male, Female, Other)	M
AgeCode	B, Y, V, S (Balak, Yuvak, Vridh, Sanyas)	Y
ElementCode	F, E, A, W (Fire, Earth, Air, Water)	F
SignCode	First letter of sun sign	L (Leo)
LocationPath	Zone.State.District.Tehsil.Village	CS.DL.5.4.1E

Example for "Rohit Mudgal": Private: X7K-RM-GM-AY-FL-CS.DL.1.A.12 | Public: X7K-RM-GM-AY-FL-CS.DL.5.4.1E

4.3 Recovery Mechanism

- **Birth-location verification** via `/recovery` endpoint — citizen answers questions about birth location
- **OTP via email/phone** using `otp_verification` table — time-limited codes
- **Password reset** without exposing private data or the Private ID

4.4 User Types and Permissions

Table 7 User Type Matrix

Type	Code	Vote	Post	Council	Admin
Human User	H_U	Yes	Yes	Yes	No
Demo User	D_U	Yes	Yes	No	No
Admin	H_U_ADMIN	Yes	Yes	Yes (Mentor)	Yes

Table 8 Age Groups (Ashrama)

Group	Code	Age Range	Council Nomination
Balak	B	0-24	No (vote if >= 13)
Yuvak	Y	25-49	Yes
Vridh	V	50-75	No (advisory)
Sanyas	S	75+	No (wisdom role)

5. Geographic Hierarchy

5.1 Hierarchy Levels

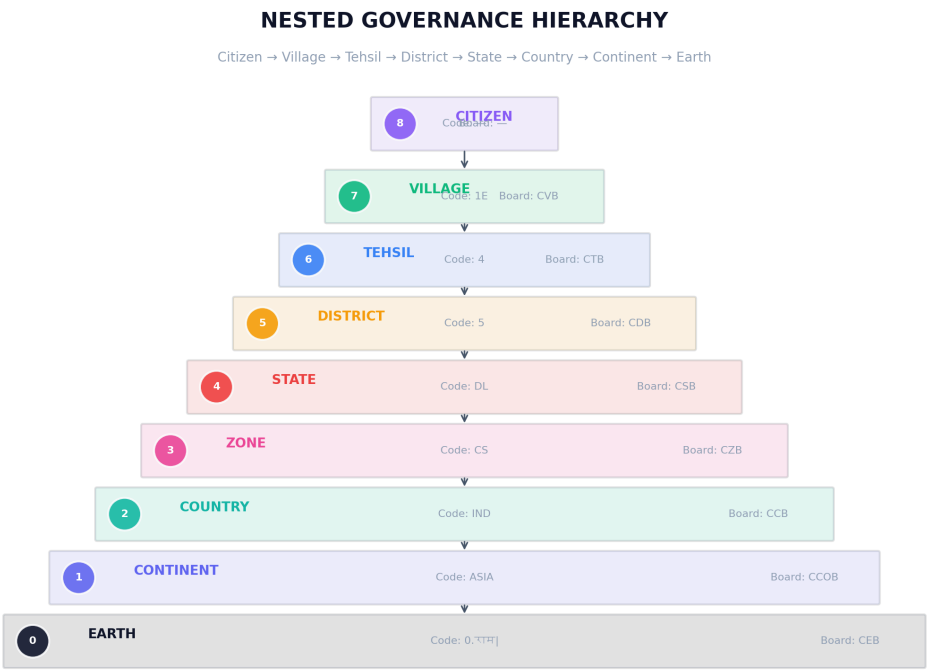


Figure 4: Eight-Level Geographic Hierarchy with codes and boards

India Zone Codes

Code	Name
CS	Central State (UT & North-East)
NS	North India State
WS	West India State
SS	South India State
ES	East India State

5.2 Collective Boards

Table 10 Collective Board Naming

Level	English	Hindi
Village	CVB	Sanyukt Gram Mandal
Tehsil	CTB	Sanyukt Tehsil Mandal
District	CDB	Sanyukt Jila Mandal
State	CSB	Sanyukt Rajya Mandal
Country	CCB	Sanyukt Desh Mandal
Continent	CCOB	Sanyukt Mahadweep Mandal
Earth	CEB	Sanyukt Prithvi Mandal

6. Four Timelines Implementation

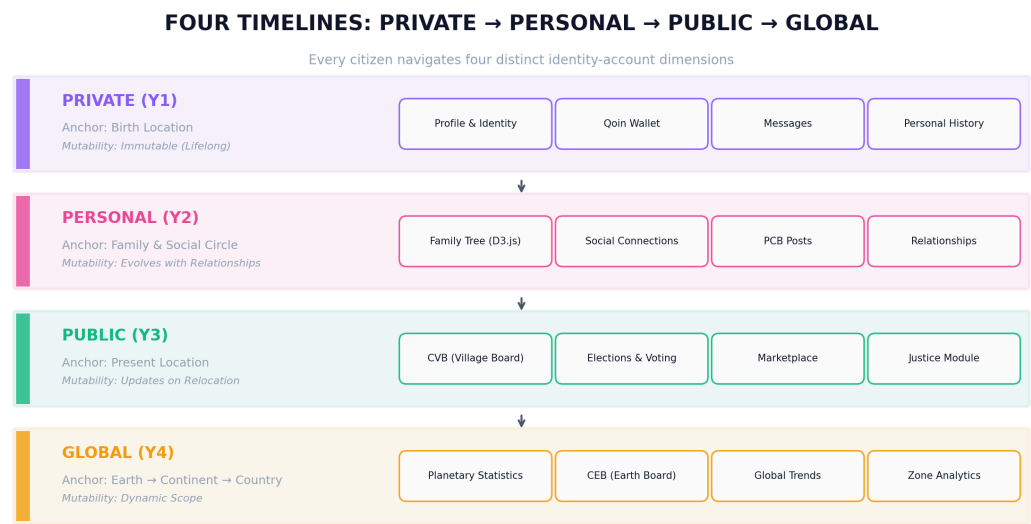


Figure 5: Four Timelines: Technical implementation of multi-dimensional identity

Table 11 Timeline Technical Specification

Timeline	Screen	Data Source	Key Features
Private (Y1)	Left top	Birth location	Profile, wallet, messages, history, birth chart
Personal (Y2)	Left middle	Family + social	PCB, posts, family tree (D3.js)
Public (Y3)	Left bottom	Present location	CVB, elections, marketplace
Global (Y4)	Left alternate	Earth → Country	Stats, CEB

The Zone tab in Global Account appears only for users with India as birth or present location. This conditional rendering ensures relevant content without cluttering the interface for international users.

7. Governance Engine

7.1 Post Escalation

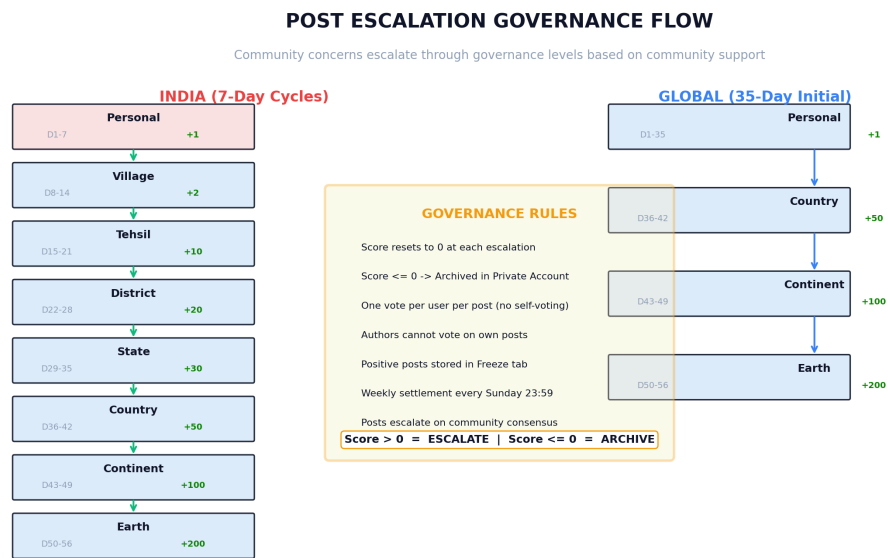


Figure 6: Post Escalation Governance Flow: India (7-day) and Global (35-day) cycles

India Escalation Schedule (7-Day Cycles)

Escalation Rules

- Score resets to 0 at each escalation level
- Score $\leq 0 \rightarrow$ post moves to Private Account "Previous Posts" (archived)
- One vote per user per post; authors cannot self-vote
- Positive posts stored in Freeze tab at each level
- Weekly settlement computes Qoin rewards every Sunday 23:59

Global Escalation (Outside India)

Table 13 Global Post Escalation Schedule

Level	Days	Next	Reward
Personal	1-35	Country	1
Country	36-42	Continent	50
Continent	43-49	Earth	100
Earth	50-56	Archive	200

7.2 Zodiac Elections

Table 14 Zodiac Mapping

Sign	Element	Code	Dates
Aries	Fire	FA	Mar 21 – Apr 19
Taurus	Earth	ET	Apr 20 – May 20
Gemini	Air	AG	May 21 – Jun 20
Cancer	Water	WC	Jun 21 – Jul 22
Leo	Fire	FL	Jul 23 – Aug 22
Virgo	Earth	EV	Aug 23 – Sep 22
Libra	Air	AL	Sep 23 – Oct 22
Scorpio	Water	WS	Oct 23 – Nov 21
Sagittarius	Fire	FS	Nov 22 – Dec 21
Capricorn	Earth	EC	Dec 22 – Jan 19
Aquarius	Air	AA	Jan 20 – Feb 18
Pisces	Water	WP	Feb 19 – Mar 20

Nomination Requirements

- Yuvak life stage (ages 25-49)
- Matching sun sign of the current election period
- Male or Female gender
- Village resident (verified through Public ID)

7.3 Council Roles

Table 15 Quantum Punch Council Roles

Role	Sanskrit	Responsibility
Mentor	Guru	Spiritual and ethical guidance for the council
Nayak	Nayak	Male elected leader
Nayika	Nayika	Female elected leader
Manager	Prabandhak	Day-to-day operations
Agent	Doot	Kiosk operator; assists citizens

7.4 Varna System

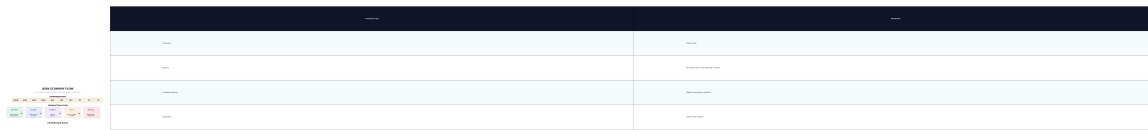
Four categories scored from verified actions:

Table 16 Varna Categories

Category	Sanskrit	Domain
Vidya	Vidya	Education, teaching
Raksha	Raksha	Governance, justice, security
Artha	Artha	Commerce, economy
Seva	Seva	Service, tree planting, community

Karma actions map to categories (e.g., `teach_hour` → Vidya, `council_day` → Raksha). The dominant category influences karma multipliers and council role recommendations.

8. Qoin Economy



...	...
...	...
...	...
...	...
...	...

Figure 8: Qoin Economy: Denominations, transaction flow, and distribution rules

8.1 Denominations

```
DENOMINATIONS = (2000, 500, 200, 100, 50, 20, 10, 5, 2, 1)
```

Rule: No single Qoin holds any other value. 7 Qoins = 5 + 2. This fixed-denomination system prevents arbitrary value inflation and ensures transparent accounting.

8.2 Greedy Algorithm

```
def min_qoins_for_amount(amount):
    qoins = []
    remaining = amount
    for denom in [2000, 500, 200, 100, 50, 20, 10, 5, 2, 1]:
        while remaining >= denom:
            qoins.append(denom)
            remaining -= denom
    return qoins
```

8.3 Weekly Settlement

- Transactions recorded in `pending_transactions` during the week
- `scheduler.py` runs `process_weekly_settlement()` every Sunday at 23:59
- Net rupee amounts converted to Qoins via greedy algorithm
- Qoins transferred from payer to payee wallets
- `weekly_statements` generated for all active users

Transaction Types

Table 17 Transaction Type Distribution

Type	Distribution
Commercial	100% to seller
Donation	50% ethical return + 50% split (20% x 5 levels)
Contribution (Karma)	Minted from governance container
Subscription	100% to kiosk container

9. Database Schema

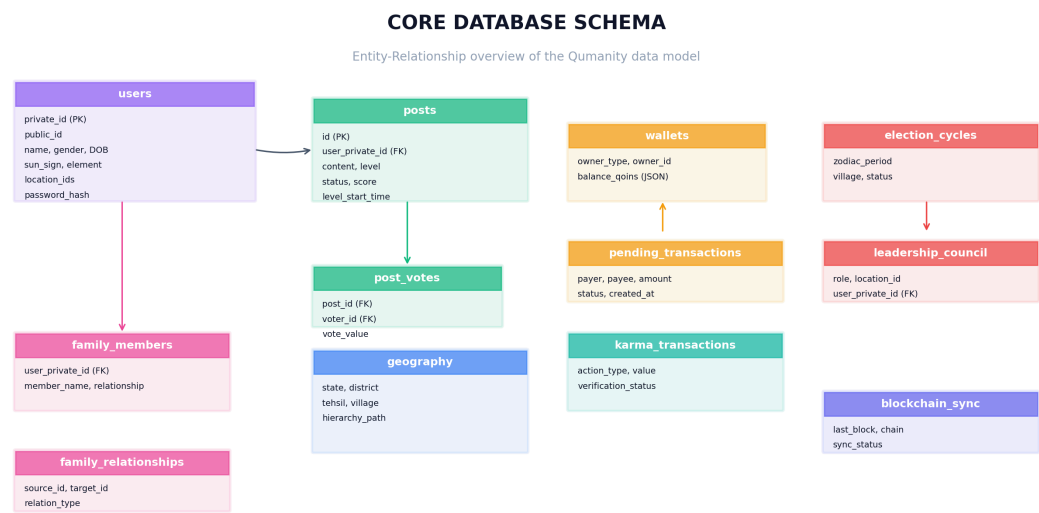


Figure 9: Core Database Entity-Relationship Overview

9.1 User Management

users (Primary Table)

Column	Type	Notes
id	INTEGER PK	Auto-increment primary key
private_id	TEXT UNIQUE	Birth-based, immutable identifier
public_id	TEXT UNIQUE	Present-location-based identifier
first_name, last_name	TEXT	Legal name
gender	TEXT	Male, Female, Male born female, Female born male
date_of_birth	DATE	Drives sun sign, age group calculation
sun_sign, moon_sign	TEXT	Calculated from DOB via ephemeris
element	TEXT	Fire, Earth, Air, Water
current_location_id	TEXT	Present village reference
birth_location_id	TEXT	Birth village reference
password_hash	TEXT	bcrypt hashed
account_type	TEXT	H_U, D_U, H_U_ADMIN
mentor_level, manager_level	INTEGER	Council role levels (default 0)
leader_level, agent_level	INTEGER	Council role levels (default 0)
is_admin	INTEGER	0 or 1
created_at	TIMESTAMP	Auto CURRENT_TIMESTAMP

Supporting User Tables

- `user_accounts` — Multiple public accounts per user (one per location)
- `user_roles` — Volunteer, Agent, Manager, Leader, Mentor assignments
- `user_education`, `user_work` — Profile history
- `otp_verification` — Email/phone OTP for recovery

9.2 Content and Social

posts

Column	Type	Notes
id	INTEGER PK	Auto-increment
content	TEXT	Post body
user_private_id	TEXT FK	References users.private_id
current_level	TEXT	personal/village/tehsil/district/state/country/continent/earth
status	TEXT	live/archived/frozen
total_score	INTEGER	Net votes (+1/-1)
level_start_time	TIMESTAMP	Escalation timer start
origin_village_id	TEXT	Author's village of origin
qoins_settled	INTEGER	Default 0

Supporting Content Tables

- `post_votes` — `post_id`, `voter_private_id`, `vote_value` (+1/0/-1)
- `messages` — Private inbox with threading
- `connection_requests` — Family/social connection requests
- `family_members`, `family_relationships` — Family tree graph

9.3 Economy

- `wallets` — `owner_type`, `owner_id`, `balance_qoins` (JSON)
- `pending_transactions` — Weekly settlement queue
- `wallet_transactions` — Settled transaction log
- `weekly_statements` — `user_private_id`, `week_start`, `week_end`, `statement_data` (JSON)
- `karma_action_types` — Predefined actions with rupee values
- `karma_transactions` — Claim records with verification status

9.4 Governance

- `election_cycles` — Zodiac period, village, status
- `election_candidates` — Nominees with approval status
- `election_votes` — Voter, candidate, timestamp
- `leadership_council` — Mentor, Nayak, Nayika, Manager, Agent per location
- `varna_profiles` — Vidya/Raksha/Artha/Seva scores per user

10. API Reference

Design Principles

- All `/api/*` routes return JSON
- `@login_required` on protected endpoints
- 401: `{"error": "Unauthorized"}`
- Exceptions: `{"error": str(e)}` — never raw tracebacks

10.1 Authentication

Table 18 Authentication Endpoints

Endpoint	Method	Purpose
<code>/register</code>	GET, POST	Registration form / submit
<code>/login</code>	GET, POST	Authentication
<code>/recovery</code>	GET	Password/ID recovery page
<code>/api/recovery/verify</code>	POST	Birth-location verification
<code>/api/otp/send</code>	POST	Send OTP to email/phone
<code>/api/otp/verify</code>	POST	Verify OTP code

10.2 Geography

Table 19 Geography Endpoints

Endpoint	Method	Purpose
/api/continents	GET	List continents
/api/countries	GET	List countries
/api/states	GET	List states
/api/states/<country_id>	GET	States by country
/api/districts	GET	Districts (filtered)
/api/tehsils	GET	Tehsils (filtered)
/api/villages	GET	Villages (filtered)
/api/geo-search	GET	Search locations

10.3 Posts and Voting

Table 20 Posts and Voting Endpoints

Endpoint	Method	Purpose
/api/posts	GET	Posts for current level
/api/post/create	POST	Create post
/api/post/vote	POST	Vote (+1/0/-1)
/api/post/escalate	POST	Trigger escalation check

10.4 Economy

Table 21 Economy Endpoints

Endpoint	Method	Purpose
/api/qoin/balance	GET	Wallet balance
/api/qoin/transfer	POST	Queue transfer (weekly)
/api/qoin/transactions	GET	Transaction history
/api/karma/claim	POST	Submit karma claim
/api/karma/status	GET	Check claim status

10.5 Elections

Table 22 Election Endpoints

Endpoint	Method	Purpose
/api/election/status	GET	Current election phase
/api/election/nominate	POST	Submit candidacy
/api/election/vote	POST	Vote for candidate
/api/admin/nomination/approve	POST	Admin approve nominee
/api/admin/nomination/reject	POST	Admin reject nominee

10.6 Family and Social

Table 23 Family and Social Endpoints

Endpoint	Method	Purpose
/api/family/tree	GET	Family tree data (D3.js)
/api/family/add_member	POST	Add family member
/api/connection/request	POST	Send connection request
/api/connection/accept	POST	Accept request

10.7 Messaging

Table 24 Messaging Endpoints

Endpoint	Method	Purpose
/api/messages/inbox	GET	Received messages
/api/messages/send	POST	Send message
/api/messages/read	POST	Mark as read

10.8 Page Routes (HTML)

Table 25 HTML Page Routes

Route	Purpose
/	Homepage
/dashboard	Four-timeline dashboard
/about, /about-details	About pages
/contact	Contact form
/calendar	Solar calendar with ephemeris
/settings	User settings
/location/<level>/<id>	Geography statistics
/admin/panel	Admin dashboard
/admin/*	Admin sub-pages

11. Core Modules

Table 26 Core Module Specification

Module	File	Responsibilities
Main App	<code>app.py</code>	Routes, session management, template rendering, API endpoints. Planned incremental split into Blueprints (<code>routes_auth</code> , <code>routes_api</code> , <code>routes_dashboard</code> , <code>routes_admin</code>).
Qoin Engine	<code>qoin_core.py</code>	Fixed denomination logic, greedy algorithm, wallet CRUD (user + nested geographic wallets), <code>pending_transactions</code> queue, <code>process_weekly_settlement()</code> , <code>weekly_statements</code> generation, karma minting, donation split logic.
Identity	<code>identity_core.py</code>	Private/Public ID generation, <code>user_accounts</code> multi-location support, OTP verification, birth-location recovery, sun sign and element calculation from DOB.
Planetary	<code>planetary_core.py</code>	Birth chart storage (Akasha/Space layer), daily ephemeris via <code>ephem</code> (sidereal Lahiri), fallback day-of-year approximation, solar calendar integration.
Varna	<code>varna_core.py</code>	Vidya/Raksha/Artha/Seva scoring, category affinity for karma actions, bonus multipliers based on dominant category, category history tracking.
Karma	<code>karma_core.py</code>	Stable import path for karma <-> varna integration, re-exports wallet helpers from <code>qoin_core</code> .
Leadership	<code>leadership_core.py</code>	Council member management, election cycle coordination, role assignment (Mentor, Nayak, Nayika, etc.).
Blockchain	<code>blockchain_core.py</code>	Idempotent schema migrations for Phase 3, <code>blockchain_sync</code> table, hash columns on <code>users</code> table, default chain: Polygon.
Language	<code>language_core.py</code> , <code>translation_s.py</code>	12 Indian language support, state-to-default language mapping, location name translations.
Donation	<code>donation_core.py</code>	Registration donation flows, donation split across nested wallets, admin donation reports.
Social	<code>social_core.py</code>	Connection requests, messaging helpers.
Scheduler	<code>scheduler.py</code>	Weekly settlement cron job, background task scheduling.
Config	<code>config.py</code>	Single source of truth for all env vars, <code>validate()</code> and <code>log_warnings()</code> at startup.

12. Planetary Ledger Node Network (PLNN)

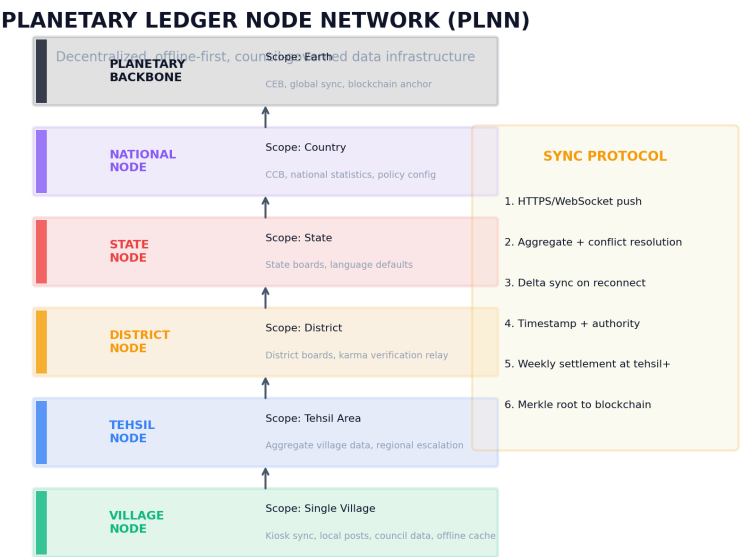


Figure 10: PLNN Node Hierarchy and Sync Protocol

12.1 Node Types

Table 27 PLNN Node Specification

Node Type	Scope	Responsibilities
Village Node	Single village	Kiosk sync, local posts, council data, offline cache
Tehsil Node	Tehsil area	Aggregate village data, regional escalation
District Node	District	District-level boards, karma verification relay
State Node	State	State boards, language defaults
National Node	Country	CCB, national statistics, policy configuration
Planetary Backbone	Earth	CEB, global sync, blockchain anchor

12.2 Sync Protocol

```

Village Node
  | periodic push (HTTPS / WebSocket)
  v
Tehsil Node
  | aggregate + conflict resolution
  v
District Node → State Node → National Node → Planetary Backbone
  |
  v
Blockchain Anchor (Polygon) – hash of weekly settlement Merkle root

```

12.3 Offline-First Strategy

1. Kiosk maintains local SQLite replica
2. Actions queued when connectivity lost
3. On reconnect, delta sync to parent node
4. Conflict resolution: timestamp + council authority at level
5. Weekly settlement computed at tehsil level minimum

12.4 Data Partitioning

Table 28 Data Partitioning Strategy

Data Type	Stored At	Synced To
User profile (private)	Birth village node	User's current village
Posts (personal)	Author's village	Escalation path nodes
Posts (public level)	Current level node	Parent on escalation
Qoin wallets	Owner's node	Parent on cross-level transfer
Elections	Village node	Tehsil (results only)
Birth charts	Birth village	Read-only to user

12.5 Status

Phase 3 — Schema and sync protocol designed; implementation not yet started. Current prototype uses single-server SQLite/PostgreSQL.

13. Security Model

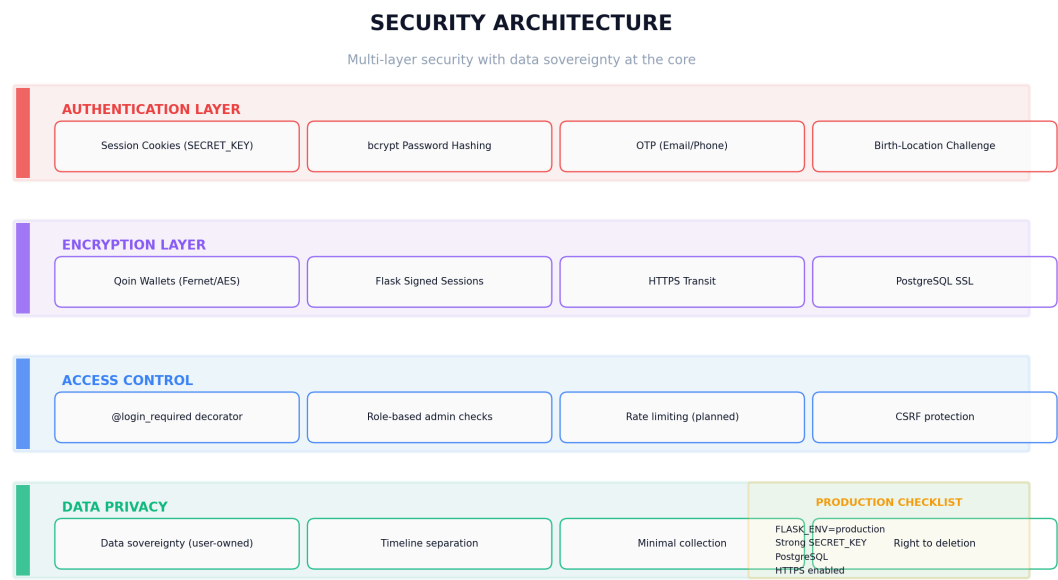


Figure 11: Multi-Layer Security Architecture

13.1 Authentication Mechanisms

Table 29 Authentication Methods

Mechanism	Usage	Implementation
Session cookies	Web login	Flask SECRET_KEY signed
bcrypt	Password hashing	12 rounds default
OTP	Email/phone recovery	Time-limited 6-digit codes
Biometric (Phase 4)	Kiosk fingerprint/face	Hardware-dependent
Birth-location challenge	ID recovery	Geo-verification questions

13.2 Encryption Layers

Table 30 Encryption Implementation

Data	Method
Qoin wallets	QOIN_WALLET_ENCRYPTION_KEY (Fernet/AES-256)
Sessions	Flask signed cookies
Passwords	bcrypt with salt
Transit	HTTPS (TLS 1.3 in production)
Database (prod)	PostgreSQL SSL connections

13.3 Container Security

- Docker runs as non-root `quantumuser`
- HEALTHCHECK polls `/health` every 30s
- No secrets in image layers — env vars only
- `.env` and `*.db` in `.gitignore`
- Read-only root filesystem where possible

13.4 API Security

- `@login_required` decorator on all protected routes
- Admin routes check `H_U_ADMIN` or role-based permissions
- Rate limiting planned (Redis-based token bucket)
- CSRF protection on all form submissions
- `data-qb-submitting` attribute prevents double-submit

13.5 Data Privacy

Table 31 Privacy Principles

Principle	Implementation
Data sovereignty	Users own data; no third-party sales
Timeline separation	Private data not exposed on Public boards
Minimal collection	Only governance-required fields
Council verification	Karma claims need local approval
Right to deletion	Deceased marking + family removal requests

13.6 Production Checklist

Table 32 Production Deployment Checklist

Item	Status	Notes
FLASK_ENV=production	Required	—
Strong SECRET_KEY (48+ char random)	Required	Use secrets.token_urlsafe(48)
Strong QOIN_WALLET_ENCRYPTION_KEY	Required	Separate from SECRET_KEY
HTTPS enabled	Required	TLS 1.3 minimum
PostgreSQL for production data	Required	Via DATABASE_URL
Persistent volume at /data	Required	For SQLite and uploads
Health check passing	Required	/health returns 200
config.validate() clean	Required	No warnings at startup
Rate limiting enabled	Recommended	Redis-based
Log aggregation	Recommended	Structured JSON logging

14. Deployment

14.1 Local Development

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python3 init_db.py
python3 app.py
# → http://127.0.0.1:5000
```

14.2 Docker

```
docker build -t qumanity .
docker run -d --name qumanity \
  -p 5000:5000 \
  -e SECRET_KEY="$(python -c 'import secrets; print(secrets.token_urlsafe(48))')" \
  -e QOIN_WALLET_ENCRYPTION_KEY="$(python -c 'import secrets; \
print(secrets.token_urlsafe(48))')" \
  -e FLASK_ENV=production \
  -v qumanity_data:/data \
  qumanity
# Or: docker compose up -d --build
```

14.3 Render Deployment

1. Push to GitHub repository
2. New → Blueprint → select repository
3. Set environment variables: SECRET_KEY, QOIN_WALLET_ENCRYPTION_KEY, FLASK_ENV=production
4. Add persistent disk at /data (if using SQLite)
5. Deploy (auto-detects Dockerfile)

14.4 Railway Deployment

1. Import GitHub repository

2. Add volume at /data
3. Set environment variables
4. Deploy via railway.json configuration

14.5 Kiosk Deployment

Table 33 Kiosk Hardware Specification

Component	Specification
Hardware	Touchscreen PC/tablet, >= 10" display
OS	Linux or locked-down browser kiosk mode
Connectivity	4G/5G or village broadband
Location	Panchayat office, community centre, market
Hours	Daylight; agent-assisted
Auth	PIN + biometric (Phase 4)
Sync	PLNN village node (Phase 3)

14.6 Database Migration (SQLite to PostgreSQL)

1. Set DATABASE_URL to PostgreSQL connection string
2. Run init_db.py against new database
3. Export SQLite data via migration script (planned)
4. Verify wallet encryption keys unchanged
5. Run integration tests on staging

15. Feature Implementation Status

15.1 Complete Features

Table 34 Implemented Features

Feature	Key Files
Registration & login	app.py, identity_core.py
Location hierarchy	indiaq.db, location.html
12 languages	translations.py, language_core.py
Posts & voting	app.py, dashboard.js
Post escalation	app.py, qoin_core.py
Qoin wallet	qoin_core.py
Weekly settlement	scheduler.py, qoin_core.py
Family tree	family_tree.js (D3.js)
Messaging	social_core.py
Admin panel	app.py, admin templates
Docker deployment	Dockerfile, docker-compose.yml

15.2 Partial Features

Table 35 Features in Development

Feature	Remaining Work
Zodiac elections	Voting UI, winner declaration
Council management	Appoint/remove, term tracking
Justice module	Formal adjudication workflow
Varna dashboard	Public-facing category display

15.3 Planned Features

Table 36 Future Feature Roadmap

Feature	Phase	Priority
Marketplace	3	High
Job portal	3	Medium
PLNN nodes	3	High
Blockchain ledger	3	Medium
iOS/macOS app	4	High
Biometric auth	4	High
Offline-first sync	4	High
AI recommendation	5	Medium
Voice interface	5	Low

16. Legal and Compliance Framework

16.1 Regulatory Positioning

Qumanity is positioned as public digital infrastructure, not a financial institution, social media company, or government body.

Table 37 Regulatory Positioning

Area	Position
Qoins	Community reward tokens, not legal tender; weekly settlement, not real-time exchange
Elections	Community advisory councils, not statutory elections
Data	User-sovereign; DPDP Act 2023 compliance target
Kiosks	Public access points, not banking correspondents

16.2 India-Specific Compliance

Table 38 India Regulatory Compliance

Regulation	Relevance	Approach
DPDP Act 2023	Personal data protection	Consent, purpose limitation, data localisation
IT Act 2000	Electronic records	Signed logs, audit trails
Panchayati Raj Act	Local governance alignment	CVB mirrors gram sabha principles
RBI guidelines	Not a payment system	Qoins are karma rewards, not e-money
Geospatial guidelines	Location data	India geospatial data stored in India

16.3 Required Legal Work (Pre-Pilot)

- DPDP compliance audit
- Terms of service and privacy policy
- Kiosk placement agreements with panchayats

- Qoin disclaimer (not legal tender)
- Data processing agreements for hosting providers

17. Budget Breakdown

17.1 Pilot Phase (50 Villages, 1 District, 12 Months)

Table 39 Pilot Phase Budget (INR)

Category	Item	Qty	Unit Cost	Total (INR)
Hardware	Kiosk (touchscreen PC + enclosure)	50	40,000-80,000	20-40 lakh
	UPS + solar backup	20	15,000	3 lakh
Connectivity	4G annual plan per kiosk	50	6,000-12,000	3-6 lakh
Development	Phase 3 dashboard + elections UI	—	—	15-25 lakh
	Phase 4 mobile app (Flutter)	—	—	20-40 lakh
	PLNN prototype (3 node types)	—	—	10-20 lakh
Personnel	Community agents (stipend)	50	3,000/mo x 12	18 lakh
	Lead developer (12 months)	1	1.5-2.5L/mo	18-30 lakh
	Governance designer (6 months)	1	1-1.5L/mo	6-9 lakh
Training	Agent + council workshops	5	1-2 lakh	5-10 lakh
Legal	DPDP audit, ToS, agreements	—	—	5-10 lakh
Hosting	Cloud (Render/Railway, 12 months)	—	—	1-3 lakh
Subtotal				1.25-2.05 crore
Contingency (15%)				15-25 lakh
Total Pilot Budget				1.4-2.3 crore INR

18. Migration Paths

18.1 SQLite to PostgreSQL

1. Set `DATABASE_URL` to PostgreSQL connection string
2. Run `init_db.py` against new database
3. Export SQLite data via migration script (planned: `scripts/migrate_sqlite_to_pg.py`)
4. Verify wallet encryption keys unchanged
5. Run integration tests on staging environment
6. Update connection pool settings in `config.py`

18.2 Version Upgrades

1. Backup database before any upgrade
2. Run schema migrations in order (`blockchain_core.py` migrations are idempotent)
3. Verify `config.validate()` passes
4. Check `/health` and `/health/details` endpoints
5. Monitor error logs for 48 hours post-upgrade

18.3 Single-Server to PLNN

1. Deploy village node software at pilot locations
2. Configure sync protocol (HTTPS/WebSocket)
3. Migrate existing data to nearest village node
4. Enable delta sync
5. Verify conflict resolution
6. Gradually decommission central server

Qumanity Technical Architecture Document v4.0

SITA Foundation (Sekyor Intelligence Tantra Foundation) | July 2026

Classification: Internal / Developer Reference | <https://qumanity.in>